

Ai programs used:

Claude vs. Gemini

(Note: Claude is set for student use purpose where Gemini isn't)

How I accessed them:

Gemini...gemini.google.com on the browser

Claude...claude.ai on windows

Prompt used:

Objective

Create a program in C or C++ that will prioritize emails for a busy company CEO.

- You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status.
- You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your email prioritizing for the CEO.

- You must create functions from scratch. Do not include pre-existing heap modules.

- Test File format:

Command: EMAIL <sender category>, <subject line>, <date>

Command Description:

The emails in the CEO's Inbox should be placed in queue based on their sender category and date. The sender categories and priority to be read are as follows:

- Boss – read first
- Subordinate – read next
- Peer – read next
- ImportantPerson – read next
- OtherPerson – read last

If there is more than one from a sender, then the newest email (not the oldest) should be read first.

EMAIL is followed by space. The rest of the fields are delimited.

Assume <sender category> is one of the five strings listed above.

Assume <subject line> is a string which may contain spaces, but not commas.

Assume <date> is in the format: MM-DD-YYYY

Command: NEXT

Command Description:

Next email for the CEO to read. Display the information on the terminal in the following format:

Sender:

Subject:

Date:

Command: READ

Command Description:

CEO has read email and dealt with it

Command: COUNT

Command Description:

Display current count of unread emails

- Your code will be tested with a different test file. The test file will begin with the emails in the CEOs Inbox, followed by the commands the CEO will use to read her Inbox. You can assume a given command will be formatted as shown, but the order and number of commands may vary. For example, be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (output should be the same for both NEXTs), two READs in a row without an intervening NEXT (highest priority email deleted from queue without displaying it).

- Sample Test File:

EMAIL Peer,Can you help me on this?,12-01-2024

EMAIL OtherPerson,Try our product,12-19-2024

EMAIL Boss,Important,12-20-2024

EMAIL Subordinate,How do I handle this?,12-25-2024

EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024

EMAIL Boss,Never Mind,01-03-2025

COUNT

NEXT

READ

NEXT

READ

COUNT

- Your program should display this:

There are 6 emails to read.

Next email:

Sender: Boss

Subject: Never Mind

Date: 01-03-2025

Next email:

Sender: Boss

Subject: Important

Date: 12-20-2024

There are 4 emails to read.

Generated Code:

Claude:

```
// email_priority.cpp
//
```

```

// Prioritizes a CEO's inbox using a MaxHeap (array/vector-based, built
from
// scratch -- no std::priority_queue or other pre-existing heap module).
//
// Priority rules:
//   1. Sender category, highest first:
//       Boss > Subordinate > Peer > ImportantPerson > OtherPerson
//   2. Within the same category, the newest email (by date) is read
first.
//
// Commands (read from stdin, one per line):
//   EMAIL <sender category>,<subject line>,<date MM-DD-YYYY>
//   NEXT    - show the next email to be read (does not remove it)
//   READ    - CEO has read/handled the current top email (removes it)
//   COUNT   - display current count of unread emails
//
// Build:  g++ -std=c++17 -O2 -o email_priority email_priority.cpp
// Run:    ./email_priority < testfile.txt

#include <iostream>
#include <sstream>
#include <string>
#include <vector>
#include <cctype>

using namespace std;

// -----
// Email record
// -----
struct Email {
    string sender;      // sender category (Boss, Subordinate, Peer, ...)
    string subject;
    string date;        // original date string, MM-DD-YYYY
    int    categoryRank; // higher = more important
    long   dateValue;    // comparable form of date, higher = newer
};

// Map a sender category string to its priority rank.
// Higher number = higher priority (read first).

```

```

int categoryRank(const string& category) {
    if (category == "Boss")            return 5;
    if (category == "Subordinate")      return 4;
    if (category == "Peer")             return 3;
    if (category == "ImportantPerson") return 2;
    if (category == "OtherPerson")      return 1;
    return 0; // unknown category - lowest priority (shouldn't occur per
spec)
}

// Convert MM-DD-YYYY into an integer YYYYMMDD so later dates compare
larger.
long dateToValue(const string& date) {
    // Expect exactly "MM-DD-YYYY"
    int mm = 0, dd = 0, yyyy = 0;
    if (sscanf(date.c_str(), "%d-%d-%d", &mm, &dd, &yyyy) == 3) {
        return static_cast<long>(yyyy) * 10000L + mm * 100L + dd;
    }
    return 0; // malformed date - treat as oldest
}

// Returns true if 'a' has strictly higher reading priority than 'b'.
bool hasHigherPriority(const Email& a, const Email& b) {
    if (a.categoryRank != b.categoryRank)
        return a.categoryRank > b.categoryRank;
    return a.dateValue > b.dateValue; // newer email wins ties
}

// -----
// MaxHeap: list(array)-based implementation, built from scratch.
// Root (index 0) always holds the highest-priority email.
// -----
class MaxHeap {
public:
    void insert(const Email& e) {
        data.push_back(e);
        heapifyUp(static_cast<int>(data.size()) - 1);
    }

    // Remove and return the highest-priority email.

```

```

Email extractMax() {
    Email top = data[0];
    data[0] = data.back();
    data.pop_back();
    if (!data.empty())
        heapifyDown(0);
    return top;
}

const Email& peekMax() const {
    return data[0];
}

bool empty() const { return data.empty(); }
size_t size() const { return data.size(); }

private:
    vector<Email> data;

    static int parent(int i) { return (i - 1) / 2; }
    static int leftChild(int i) { return 2 * i + 1; }
    static int rightChild(int i) { return 2 * i + 2; }

    void heapifyUp(int i) {
        while (i > 0 && hasHigherPriority(data[i], data[parent(i)])) {
            swap(data[i], data[parent(i)]);
            i = parent(i);
        }
    }

    void heapifyDown(int i) {
        int n = static_cast<int>(data.size());
        while (true) {
            int largest = i;
            int l = leftChild(i);
            int r = rightChild(i);

            if (l < n && hasHigherPriority(data[l], data[largest]))
                largest = l;
            if (r < n && hasHigherPriority(data[r], data[largest]))

```

```

        largest = r;

        if (largest == i) break;

        swap(data[i], data[largest]);
        i = largest;
    }
}

};

// -----
// Helpers for parsing input lines
// -----
static string trim(const string& s) {
    size_t start = s.find_first_not_of(" \t\r\n");
    if (start == string::npos) return "";
    size_t end = s.find_last_not_of(" \t\r\n");
    return s.substr(start, end - start + 1);
}

// Split "EMAIL <category>,<subject>,<date>" into its three fields.
// Subject may contain spaces but never commas, so a simple split on
// commas (into exactly 3 parts, first two split, remainder is date)
// works.
static bool parseEmailLine(const string& rest, Email& outEmail) {
    // rest is everything after "EMAIL " (already trimmed of leading
    // spaces)
    size_t firstComma = rest.find(',');
    if (firstComma == string::npos) return false;

    size_t secondComma = rest.rfind(','); // date has no commas, subject
    has none either,
                                         // so the LAST comma separates
    subject from date
    if (secondComma == string::npos || secondComma == firstComma) {
        // exactly one comma found but we need two fields separators;
        // handle case where subject itself might be empty etc.
        secondComma = rest.find(',', firstComma + 1);
        if (secondComma == string::npos) return false;
    }
}

```

```

        string category = trim(rest.substr(0, firstComma));
        string subject  = trim(rest.substr(firstComma + 1, secondComma -
firstComma - 1));
        string date      = trim(rest.substr(secondComma + 1));

        outEmail.sender    = category;
        outEmail.subject    = subject;
        outEmail.date       = date;
        outEmail.categoryRank = categoryRank(category);
        outEmail.dateValue   = dateToValue(date);
        return true;
    }

// -----
// Main: read commands from stdin, drive the MaxHeap
// -----
int main() {
    MaxHeap inbox;
    string line;

    while (getline(cin, line)) {
        string trimmed = trim(line);
        if (trimmed.empty()) continue;

        if (trimmed.rfind("EMAIL", 0) == 0) {
            // "EMAIL" followed by a space, then the delimited fields
            string rest = trimmed.substr(5); // strip "EMAIL"
            rest = trim(rest);
            Email e;
            if (parseEmailLine(rest, e)) {
                inbox.insert(e);
            } else {
                cerr << "Warning: could not parse EMAIL line: " << line <<
endl;
            }
        }
        else if (trimmed == "NEXT") {
            if (inbox.empty()) {
                cout << "Next email:" << endl;
            }
        }
    }
}

```

```

        cout << "    No emails in inbox." << endl;
    } else {
        const Email& e = inbox.peekMax();
        cout << "Next email:" << endl;
        cout << "    Sender: " << e.sender << endl;
        cout << "    Subject: " << e.subject << endl;
        cout << "    Date: " << e.date << endl;
    }
}

else if (trimmed == "READ") {
    if (inbox.empty()) {
        cout << "No emails to read." << endl;
    } else {
        inbox.extractMax();
    }
}

else if (trimmed == "COUNT") {
    cout << "There are " << inbox.size() << " emails to read." <<
endl;
}

else {
    cerr << "Warning: unrecognized command: " << line << endl;
}

}

return 0;
}

```

Gemini:

```

#include <iostream>
#include <string>
#include <sstream>
#include <vector>
#include <iomanip>

struct Email {
    std::string sender;
    std::string subject;
    std::string dateStr; // Format: MM-DD-YYYY

```



```

int priorityCategory;
long long numericDate; // For chronological ordering (YYYYMMDD)

// Parse date into YYYYMMDD for easy numerical comparison
void parseDate() {
    int m, d, y;
    char dash1, dash2;
    std::stringstream ss(dateStr);
    ss >> m >> dash1 >> d >> dash2 >> y;
    numericDate = static_cast<long long>(y) * 10000 + m * 100 + d;
}

// Determine category weight: Boss (5) > Subordinate (4) > Peer (3) >
ImportantPerson (2) > OtherPerson (1)
void parsePriority() {
    if (sender == "Boss") priorityCategory = 5;
    else if (sender == "Subordinate") priorityCategory = 4;
    else if (sender == "Peer") priorityCategory = 3;
    else if (sender == "ImportantPerson") priorityCategory = 2;
    else priorityCategory = 1; // OtherPerson
}

// Returns true if this email has HIGHER priority than 'other'
bool operator>(const Email& other) const {
    if (priorityCategory != other.priorityCategory) {
        return priorityCategory > other.priorityCategory;
    }
    // Newer date takes precedence over older date
    return numericDate > other.numericDate;
}
};

```

```

class MaxHeap {
private:
    std::vector<Email> heap;

    void heapifyUp(int index) {
        while (index > 0) {
            int parent = (index - 1) / 2;
            if (heap[index] > heap[parent]) {

```

```

        std::swap(heap[index], heap[parent]);
        index = parent;
    } else {
        break;
    }
}

}

void heapifyDown(int index) {
    int size = heap.size();
    while (index < size) {
        int left = 2 * index + 1;
        int right = 2 * index + 2;
        int largest = index;

        if (left < size && heap[left] > heap[largest]) {
            largest = left;
        }
        if (right < size && heap[right] > heap[largest]) {
            largest = right;
        }

        if (largest != index) {
            std::swap(heap[index], heap[largest]);
            index = largest;
        } else {
            break;
        }
    }
}

public:
    void insert(const Email& email) {
        heap.push_back(email);
        heapifyUp(heap.size() - 1);
    }

    bool peek(Email& result) const {
        if (heap.empty()) return false;
        result = heap[0];
    }

```

```

        return true;
    }

    bool pop() {
        if (heap.empty()) return false;
        heap[0] = heap.back();
        heap.pop_back();
        if (!heap.empty()) {
            heapifyDown(0);
        }
        return true;
    }

    size_t size() const {
        return heap.size();
    }

    bool empty() const {
        return heap.empty();
    }
};

// Trim leading whitespace helper
std::string trim(const std::string& str) {
    size_t start = str.find_first_not_of(" \t");
    return (start == std::string::npos) ? "" : str.substr(start);
}

int main() {
    MaxHeap emailQueue;
    std::string line;

    while (std::getline(std::cin, line)) {
        if (line.empty()) continue;

        if (line.rfind("EMAIL", 0) == 0) {
            std::string payload = trim(line.substr(5));
            std::stringstream ss(payload);
            std::string sender, subject, date;

```

```

        if (std::getline(ss, sender, ',') &&
            std::getline(ss, subject, ',') &&
            std::getline(ss, date)) {

            Email email;
            email.sender = trim(sender);
            email.subject = trim(subject);
            email.dateStr = trim(date);
            email.parsePriority();
            email.parseDate();

            emailQueue.insert(email);
        }
    }
    else if (line == "COUNT") {
        std::cout << "There are " << emailQueue.size() << " emails to
read." << std::endl;
    }
    else if (line == "NEXT") {
        Email nextEmail;
        if (emailQueue.peek(nextEmail)) {
            std::cout << "Next email:" << std::endl;
            std::cout << "    Sender: " << nextEmail.sender <<
std::endl;
            std::cout << "    Subject: " << nextEmail.subject <<
std::endl;
            std::cout << "    Date: " << nextEmail.dateStr <<
std::endl;
        }
    }
    else if (line == "READ") {
        emailQueue.pop();
    }
}

return 0;
}

```

Analysis:

1. Correctness

Both programs run mostly without any issues with the given test file and other test cases. They also implement the array/vector base MaxHeap correctly. Claude's version is more robust, handles edge cases better, and is transparent if there are any errors (malformed input) or a command that cannot be executed (read when no email). Gemini's version silently fails. Gemini's version also doesn't guarantee that it would work on another OS than Windows (what I'm using), since they don't trim the trailing white spaces.

2. Execution Time

Gemini does better when popping the root of the heap. Claude copies all of the Email to return in the `extractMax()` but it serves no purpose when using than popping (READ).

3. Space Complexity

Identical for both programs

4. Maintainability

Both Claude and Gemini have comments, however Claude takes it a step further and adds in a header comment that explains the rules of the program that is written, how to build and run the program, and is more descriptive in the inline comments. Claude also neatly aligns some of its code to make it easier to read. One problem that I didn't like about Claude's code, is that sometimes it would skip putting `}` after if statements. It doesn't make a big difference when it comes to running the program, but it could introduce some bad bugs if the person who edits it later doesn't pick that up. There were also modules that Claude doesn't use and code that is just dead weight.

Code base: Claude's version

It is as good or better than Gemini in almost every single way.

Edits:

1. Correctness

This was my fault when I prompted, but there are no newlines between the outputs. So I added them in. I took out modules that the program wasn't using `<sstream>` and `<cctype>`. I also made an improvement in the parsing logic by taking out a meaningless checking condition and action.

```
if (secondComma == string::npos || secondComma == firstComma) {  
    // exactly one comma found but we need two fields separators;  
    // handle case where subject itself might be empty etc.  
    secondComma = rest.find(',', firstComma + 1); // dead weight  
    if (secondComma == string::npos) return false;  
}
```

2. Execution Time

I simplified the `extractMax()` down to a simple pop, since we don't need the functionality of returning the values of the email we just popped.

3. Space Complexity

No meaningful changes can be made.

4. Maintainability

Added in all the {} after the if statements that were missing. I also added in comments for every line of code. Changed the date values to be ints rather than longs since, we don't need that big of numbers for dates (even with int, we still can have number 20times bigger than the largest date we can parse)